INTERNATIONAL EXHIBITION | PROFESSIONAL JURY SUBMISSION

LeakShield Pro

Complete Product, Architecture and Security Documentation

A free and open-source cybersecurity assessment platform that discovers public security risks, presents exact evidence, teaches secure development, and guides remediation.

DOCUMENT	PLATFORM	STATUS
Jury Edition 2026	Web + Code Security	Production Ready

Enterprise-quality cybersecurity for everyone, completely free.

Live: leak-shield-pro-2-0.vercel.app
Repository: github.com/shaheerali4/leak-sheild-pro-2-0
Prepared: 22 August 2026

DOCUMENT CONTROL

About This Report

A current, jury-ready explanation of what LeakShield Pro does, why it exists, how it works, how it stays safe, and where it can grow.

Item	Detail
Project	LeakShield Pro
Category	Cybersecurity, DevSecOps and secure software education
Primary users	Developers, students, startups, educators and small businesses
Assessment scope	Authorized public websites, pasted text, configuration, CI logs and project folders
Deployment	Vercel production deployment and Docker/local development
Commercial dependency	None required. Core operation uses free and public technology.
Safety position	Defensive, bounded and low-impact. No credential guessing or exploit execution.

How to use this report

Read the Executive Summary for the project pitch, the Workflow and Architecture sections for technical understanding, the Security Model for trust and safety, and the Exhibition Runbook for the live jury demonstration.

Contents

01 | EXECUTIVE SUMMARY

Security Evidence That Developers Can Act On

LeakShield Pro transforms raw public website and code signals into clear, defensible and educational security findings.

DISCOVER
Find public attack-surface routes, configuration weaknesses, exposed files, technology signals and potential secrets.

PROVE
Show the exact URL, file, line, column, header, cookie or component supporting each result.

PRIORITIZE
Combine severity, confidence, evidence quality and public exposure into understandable risk.

REMEDIATE
Explain business impact and provide step-by-step fixes with trusted official references.

The core problem

Professional security tools can be expensive, traditional reports are difficult for beginners, and vague warnings do not help a developer prove or repair the root cause. LeakShield Pro addresses all three gaps in one free platform.

The project outcome

A user can submit an authorized public URL, paste security-relevant text, or provide a project folder. The platform validates the input, performs bounded assessment phases, produces grouped findings, and connects every useful result to evidence, explanation and remediation.

Jury value proposition

This is not a page that displays random warnings. It is an end-to-end security workflow: acquisition, validation, evidence collection, deduplication, risk scoring, education, remediation, session-scoped history, reporting, testing and free deployment.

02 | PURPOSE

Problem Statement, Audience and Principles

The platform is built around practical security access: accurate enough to be trusted, understandable enough to teach, and free enough to reach everyone.

Who benefits

Developers
Check public deployment posture and receive implementation-focused fixes.

Students
Learn why a weakness matters and how secure design prevents it.

Startups
Receive an initial security roadmap without a mandatory commercial scanner.

Educators and teams
Demonstrate responsible assessment and evidence-based reporting.

Product principles

+ Evidence before claims - every useful result should identify what was observed and where.
+ Safe by design - public targets only, bounded requests, redirect revalidation and no exploit payloads.
+ Education with action - explain the weakness and connect it to a practical fix.
+ Free core operation - no mandatory Shodan, paid AI model, paid API or commercial subscription.
+ Transparent limitations - distinguish confirmed evidence, potential indicators and authorization-required tests.
+ Production engineering - typed data, modular engines, automated testing, secure configuration and maintainable code.

03 | WORKFLOW

Complete Start-to-End User Journey

From the shield unlock to remediation, each stage has one clear purpose.

Detailed sequence

01 The shield unlock animation introduces the security console and loads the main application.

02 The user opens Scan and selects website, text or project-folder mode.

03 The backend validates type, size, public address, ownership header and safety limits.

04 Assessment engines collect HTTP, DNS, TLS, JavaScript, configuration and code evidence as applicable.

05 Detection rules normalize, deduplicate and redact potentially sensitive results.

06 The risk engine calculates severity, confidence and contextual priority.

07 The Findings interface groups related results and exposes exact evidence only when opened.

08 Learning Mode, framework fixes, roadmap and report export guide the remediation process.

09 Mission Archive stores session-owned history that the user can reopen or clear.

Important boundary

Finding a route, technology or form does not automatically mean it is vulnerable. LeakShield labels discovery information separately from direct evidence and from checks that require explicit authorization.

04 | CAPABILITIES

Website and Code Assessment Coverage

LeakShield combines multiple free signals while keeping every operation bounded and defensible.

Module	What is checked	Evidence type
Discovery	Same-origin pages, robots.txt, sitemap.xml, common routes and JavaScript links	Attack-surface inventory
Headers	CSP, HSTS, framing, MIME, referrer, permissions and browser isolation	Observed response configuration
TLS	Certificate hostname, issuer, expiry, protocol and cipher	Negotiated public TLS evidence
DNS	A/AAAA, MX, TXT, SPF, DMARC, DKIM, CAA, NS, CNAME and DNSSEC	Public DNS records
Subdomains	Certificate Transparency plus DNS resolution and HTTP reachability	Free public CT and network signals
Technology	Headers, HTML, scripts, cookies and product/version fingerprints	Wappalyzer-style evidence
Exposure	Git, env, backup, SQL, archive, config, debug and directory-index signals	Public response verification
Secrets	Cloud keys, API keys, tokens, JWTs, private keys and connection strings	Redacted exact location
Web indicators	Forms, CSRF visibility, uploads, CORS, cookies, redirects, SQL errors and DOM-XSS signals	Passive or low-impact evidence
CVE	Exact product and version correlation through NIST NVD	Known-vulnerability intelligence
Network	Small bounded common-port checks and RDAP ownership	Public service context

05 | ACCURACY

Exact Evidence and Verification Model

Professional security output must explain what happened, where it happened, what proves it, and how to fix it.

WHAT
The vulnerability or security condition identified by the rule.

WHERE
Exact URL, file, line, column, header, cookie or affected component.

PROOF
Observed redacted evidence, expected secure state and detection method.

ACTION
Business impact, remediation, framework guidance and trusted references.

Verification states

State	Meaning	How the user should respond
Confirmed	Direct public evidence supports the result.	Prioritize according to severity and remediate.
Potential	A relevant pattern exists but requires human verification.	Inspect context before treating it as a proven vulnerability.
Informational	Useful surface or configuration information, not a proven flaw.	Use it to understand exposure and plan hardening.
Authorization required	Active confirmation would require permission and is not attempted.	Use a separate controlled assessment after written authorization.

Secret evidence without spreading the secret

A potential key can be shown as a short redacted preview such as `AIza...50Us`, together with the exact public resource and line/column location. The complete credential is not repeated in the dashboard, report or audit record.

Why this matters
Severity and confidence are different. A potentially critical impact does not become confirmed evidence merely because the possible damage is high.

06 | ARCHITECTURE

Clean, Modular and Deployment-Friendly Design

The system separates user experience, request validation, assessment, risk, education and persistence so each part can evolve safely.

STRUCTURED FINDINGS -> GROUPED DASHBOARD -> ACTIONABLE ROADMAP

Exact evidence, risk context, remediation and official references

Layer responsibilities

Layer	Responsibility
React dashboard	Input modes, progress, grouped findings, evidence inspection, themes, history and reports.
FastAPI routes	Typed validation, ownership controls, rate limits and stable JSON endpoints.
Scan Service	Coordinates safety checks, assessment engines, risk, explanation, cache and persistence.
Assessment engines	Collect bounded website, network, DNS, TLS, technology and exposure signals.
Detection engine	Runs rule modules, maps exact locations, deduplicates and redacts values.
Risk and education	Prioritizes evidence and produces understandable impact, remediation and Learning Mode content.
Data layer	Stores scans/findings in PostgreSQL or local SQLite and optionally caches repeated work in Redis.

07 | EXPERIENCE

Focused Interface for Scanning and Findings

The interface keeps the dramatic shield opening, then shifts to a clean professional workspace designed for quick decisions.

SHIELD INTRO
A branded unlock sequence creates a memorable security identity without changing scan logic.

CONSOLE
The overview presents the latest grade, score, important metrics and primary action.

SCAN
Threat acquisition accepts authorized inputs and Mission Archive manages session history.

FINDINGS
Related results are grouped under expandable sections with an exact evidence inspector.

Supporting information without page overload

Attack surface, CVE intelligence, report export and the security field guide live inside collapsible drawers under Findings. This keeps the main navigation limited to Scan and Findings while preserving important capabilities.

Accessibility and compatibility

+ Responsive layouts for desktop, tablet and mobile devices.
+ Professionally designed dark and light themes.
+ Keyboard-friendly controls and visible interaction states.
+ Expandable sections that open independently without stretching empty neighboring cards.
+ Compact grouping that prevents long, unprofessional result walls.

08 | INTELLIGENCE

Risk Scoring, Learning Mode and Fix Guidance

The platform moves from detection to decision support by explaining both urgency and implementation.

Risk model

The risk engine combines rule severity, confidence, evidence quality, public exposure and contextual signals into a 0-100 score and grade. Findings remain transparent about uncertainty instead of converting weak evidence into absolute claims.

Learning Mode

+ Definition and beginner-friendly explanation
+ Why the issue is dangerous and its business impact
+ Generalized attacker behavior without exploit instructions
+ Common developer mistakes and secure coding recommendations
+ Step-by-step remediation and prevention checklist
+ OWASP, MDN, RFC, MITRE, NIST or official vendor references

Developer Fix Assistant

When relevant, findings can include safe configuration or coding recommendations for Apache, Nginx, Express.js, Next.js, React, Laravel, Node.js, Spring Boot and generic frameworks. Exploit code is intentionally excluded.

Improvement roadmap

Priority	Typical meaning	Planning output
Priority 1	Critical exposure with strong evidence	Immediate containment, rotation or access restriction
Priority 2	High-risk weakness	Near-term engineering fix and verification
Priority 3	Medium-risk hardening	Scheduled remediation with owner and effort
Priority 4	Low or informational improvement	Backlog hardening and monitoring

09 | TRUST

Platform Security, Privacy and Safe-Scanning Controls

A cybersecurity platform must protect both its users and the systems it assesses.

SSRF DEFENSE
Blocks loopback, private, reserved and local addresses before requests and after redirects.

BOUNDED WORK
Limits file size, project size, response size, crawl breadth, ports and execution time.

SESSION ISOLATION
Hashes a random browser session identifier and scopes list, detail and delete operations to its owner.

SECRET REDACTION
Stores and displays hashes, previews and safe context rather than full credential values.

Additional controls

+ Typed Pydantic input and response validation.
+ Rate limiting on important public and admin routes.
+ CORS and trusted-host configuration.
+ Signed admin sessions with environment-based account allowlisting.
+ No password guessing, exploit submission, authorization bypass or dangerous file upload.
+ No real credentials stored in source code or project documentation.

Admin access
The admin portal is intentionally absent from normal navigation and is opened through /admin=true. Real accounts and the session-signing secret belong only in protected deployment environment variables.

10 | OPEN SOURCE

Free Data Sources and No Mandatory Paid Services

The project philosophy is practical: enterprise-quality security assistance should remain available without a commercial dependency.

What the platform uses

Source	Purpose
Direct HTTP/HTTPS	Response status, headers, HTML, scripts, cookies and public files
DNS	Public host, mail and domain-security records
TLS	Certificate and negotiated encryption information
crt.sh	Free public Certificate Transparency subdomain data
RDAP	Free public network ownership and registration context
NIST NVD	Free exact-version known-vulnerability correlation
Local rules	Deterministic detection, scoring, explanation and education

What the platform does not require

NO SHODAN

LeakShield Pro does not use a Shodan API key for its assessment workflow.

NO PAID AI

Core advisor and education content works through local deterministic rules.

NO PREMIUM DATABASE

PostgreSQL, SQLite and Redis are open technologies with free deployment options.

NO COMMERCIAL SCANNER

The core platform remains functional without a mandatory proprietary service.

11 | ENGINEERING

Technology Stack and Codebase Map

The stack is intentionally modern, typed where practical, modular and suitable for free hosting.

Area	Technology	Purpose
Frontend	React 18, Vite, Tailwind/project CSS, Lucide	Responsive dashboard, themes, components and production bundle
Backend	Python 3.11, FastAPI, Pydantic	Async API, input validation and OpenAPI documentation
Assessment	HTTPX, dnspython, TLS sockets, local rules	Bounded public signal collection and analysis
Database	SQLAlchemy Async, PostgreSQL, SQLite	Durable production records and local fallback
Cache	Redis	Optional repeated-result acceleration
Deployment	Vercel, Docker Compose	Public production hosting and reproducible local stack
Quality	Pytest, Ruff, Bandit, pip-audit, npm audit	Behavior, style, security and dependency verification

Repository map

Path	Responsibility
backend/app/api	Scan and admin HTTP routes
backend/app/engines	Website assessment, detection, risk and education
backend/app/services	Complete scan orchestration
backend/tests	Backend behavior and security tests
frontend/src/components	Console, scanner, findings, admin, knowledge and shield UI
frontend/src/data	Knowledge-base content
docs	Architecture, audit, report and user documentation
scripts	Reproducible report and project utilities
.github/workflows	Automated security and quality pipeline
vercel.json / docker-compose.yml	Production and local deployment configuration

12 | OPERATIONS

API, Configuration and Deployment Workflow

The same shared FastAPI implementation supports local Docker and the Vercel production workflow.

Primary API endpoints

Method	Endpoint	Purpose
GET	/api/health	Confirm service availability
POST	/api/scans	Start a website, text or project-folder scan
GET	/api/scans	List scans owned by the current session
GET	/api/scans/{id}	Load one owned scan and its findings
DELETE	/api/scans	Clear the current session's scan history
POST	/api/admin	Authenticate an allowed administrator
GET / DELETE	/api/admin	Read or clear protected admin audit information

Important environment settings

Production configuration includes `DATABASE_URL`, `REDIS_URL`, `CORS_ORIGINS`, `ALLOWED_HOSTS`, `ADMIN_ACCOUNTS_JSON` and `ADMIN_SESSION_SECRET`. Real values must never be committed.

Deployment sequence

01 Implement the change locally on a feature branch.

02 Run backend, frontend, dependency and security checks.

03 Commit only intended files and push to GitHub.

04 Merge the verified commit into main.

05 Vercel builds the connected repository and publishes the update.

06 Verify the live browser workflow and production API response.

13 | ASSURANCE

Testing, Review and Production Quality

Security features are supported by automated checks rather than visual claims alone.

BACKEND TESTS
Pytest covers services, routes, rules, ownership, validation and security behavior.

STATIC QUALITY
Ruff and Bandit identify Python correctness, style and insecure coding patterns.

DEPENDENCIES
pip-audit and npm audit check known package vulnerabilities.

FRONTEND BUILD
Vite production build confirms that the deployed interface compiles successfully.

Automated GitHub pipeline

+ Root Node API compatibility tests
+ Frontend dependency audit and production build
+ Python Ruff linting and Bandit security analysis
+ Python dependency audit and package consistency check
+ Complete backend Pytest suite

Quality rule
Warnings are investigated instead of silently suppressed. Security fixes target root causes and preserve existing behavior whenever possible.

14 | EXHIBITION

Professional Jury Demonstration Runbook

A short, evidence-focused demonstration communicates more value than opening every module.

Time	Scene	Jury message
00:00-00:15	Launch	Show the shield unlock and name the platform.
00:15-00:35	Problem	Explain that developers need affordable and understandable security evidence.
00:35-01:10	Scan	Enter a team-owned or authorized URL and start the assessment.
01:10-01:40	Evidence	Open one grouped finding and point to URL, proof, confidence and verification state.
01:40-02:05	Remediation	Show Learning Mode, framework guidance and improvement roadmap.
02:05-02:25	Trust	State: passive checks, no Shodan, no paid API, secrets redacted.
02:25-02:45	Close	Show report/export and present the future scope.

Best demonstration target

Use a deliberately vulnerable local lab, a project-owned website, or a target with explicit written authorization. Never expose a real API key, admin password or private customer website during recording or exhibition.

Strong jury proof points

+ The finding includes exact evidence instead of only a vulnerability name.
+ The platform separates confirmed, potential, informational and authorization-required results.
+ Sensitive values are redacted while their public location remains provable.
+ The same result includes business impact, secure fix and learning material.
+ The project remains functional without Shodan or a paid AI model.

15 | ROADMAP

Transparent Limitations and Future Scope

Trust grows when a platform clearly states what it can prove today and what belongs in a future authorized testing environment.

Current limitations

+ The Vercel runtime cannot execute every large native cybersecurity binary.
+ Public website assessment is intentionally bounded to protect targets and hosting resources.
+ Passive evidence cannot confirm every SQL injection, XSS, authentication, authorization, upload or business-logic flaw.
+ External DNS, TLS, Certificate Transparency and NVD services can be temporarily unavailable.
+ Technology versions can be hidden by proxies, CDNs or custom headers.
+ Secret-pattern results can include intentionally public identifiers and require human verification.
+ Session history is not yet a complete multi-user workspace and role system.

Future roadmap

Horizon	Planned direction
Near term	Target ownership proof, improved comparisons, notifications and expanded framework guidance
Team edition	User accounts, shared workspaces, role-based access and signed report verification
Developer workflow	GitHub pull-request checks, CI/CD integration and scheduled monitoring
Self-hosted lab	Optional Katana, Subfinder, Nuclei and OWASP ZAP adapters in isolated containers
Advanced coverage	Authenticated assessment, API/GraphQL modules and controlled dynamic analysis
Education	Multilingual learning content and community-contributed rules

16 | REFERENCE

Simple Technical Glossary

Plain-language definitions help technical and non-technical jury members understand the same project.

Term	Easy meaning
API	A controlled way for two software systems to communicate.
Attack surface	All public routes, services and components that may be reached.
CORS	Browser rules controlling which other websites can read a response.
CSP	Browser rules controlling which scripts and resources may load.
CVE	A public identifier for a known software vulnerability.
CWE	A category describing a type of software weakness.
Evidence	The exact observed information supporting a finding.
False positive	A warning that appears dangerous but is not a valid issue.
Remediation	The practical steps used to fix a security weakness.
Risk severity	How much damage a valid issue could cause.
Confidence	How strongly the available evidence supports the result.
SSRF	A weakness that tricks a server into requesting a protected address.
TLS/SSL	The encryption that protects HTTPS traffic.
Redaction	Hiding the sensitive middle of a secret value.

17 | JURY DEFENSE

Questions the Team Should Be Ready to Answer

Short, accurate answers demonstrate technical maturity and responsible security engineering.

Is every finding a confirmed vulnerability?

No. The platform labels confirmed evidence, potential indicators, informational surface and authorization-required checks separately.

Does it attack the website?

No. Core website assessment is passive or low-impact and does not submit exploit payloads, guess credentials or bypass access control.

How do you prove a result?

The finding includes the exact resource, line or component, redacted evidence, expected state, method and verification status.

Does it use Shodan or paid AI?

No. Core operation uses free public HTTP, DNS, TLS, crt.sh, RDAP, NIST NVD and local deterministic rules.

Why call it AI-powered?

Its advisor converts structured evidence into context-aware summaries, priorities and education. The core remains deterministic and does not depend on a paid LLM.

How is the scanner itself secured?

SSRF controls, redirect revalidation, strict limits, typed validation, rate limiting, session ownership, redaction and signed admin sessions.

What makes it different from a simple scanner?

It combines discovery, proof, prioritization, learning, framework fixes, roadmap, history and reporting in one workflow.

18 | CONCLUSION

A Security Platform That Finds, Explains and Helps Fix

LeakShield Pro is designed to make responsible cybersecurity evidence available to the people who need it most.

Enterprise-quality cybersecurity for everyone, completely free.

The project's value is not limited to detecting a technical pattern. It creates a full chain from authorized input to defensible evidence, risk understanding, secure remediation and long-term learning. That combination makes LeakShield Pro suitable for developers, classrooms, startups, small businesses and an international technology exhibition.

Live platform
https://leak-shield-pro-2-0.vercel.app/

Open-source repository
https://github.com/shaheerali4/leak-sheild-pro-2-0

Prepared from the current LeakShield Pro implementation and complete project documentation. No real credentials are included in this report.